

Algoritmos e Introdução a Python

Profº: Joseph Donald



(83) 98228-8607



@josephdonald



030106382@prof.uninassau.edu.br



UNINASSAU



ser
educacional



Veritas

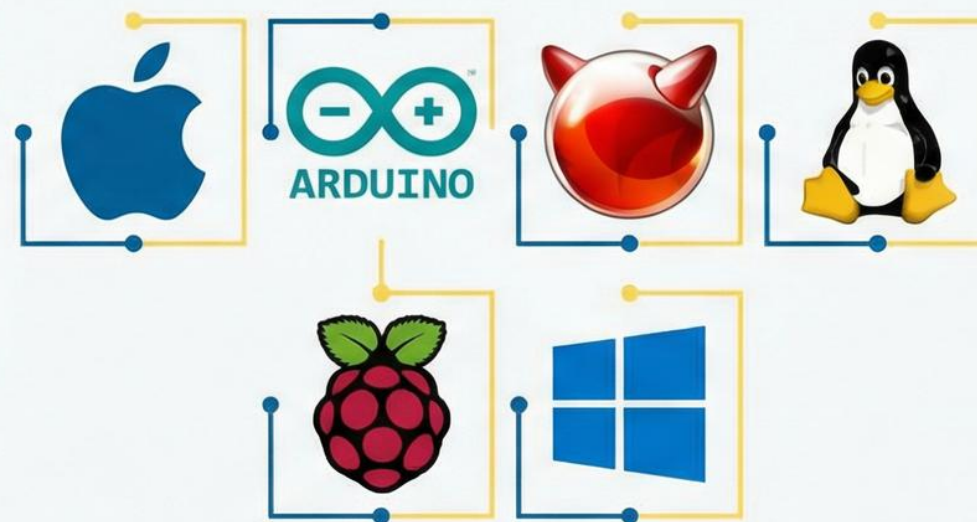
Por que Python?

Gratuita

- O Python é um software livre e pode ser utilizado de forma totalmente gratuita, graças ao trabalho contínuo e colaborativo da Python Foundation (www.python.org).
- Sua versatilidade permite que utilizemos a linguagem em praticamente qualquer arquitetura ou sistema operacional disponível no mercado.

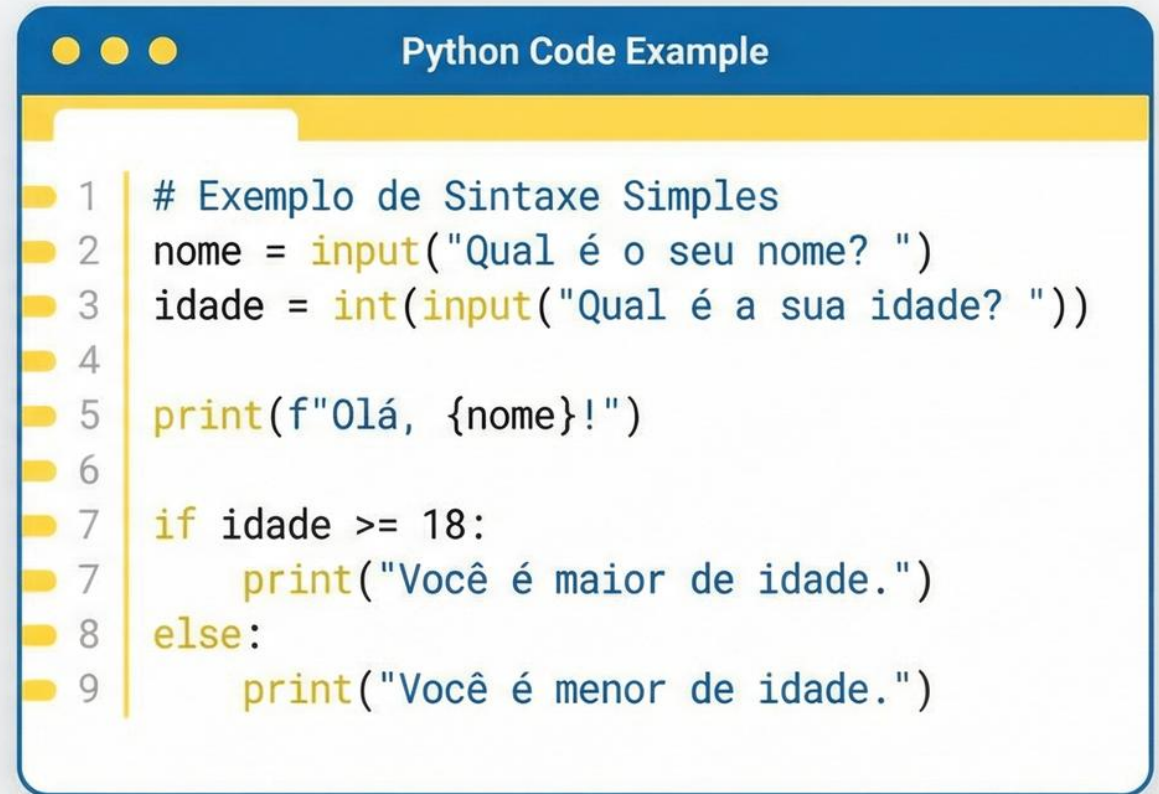
Sistemas suportados incluem plataformas como Raspberry, Arduino, distribuições Linux, FreeBSD, Microsoft Windows e MacOS X.

Sistemas Suportados



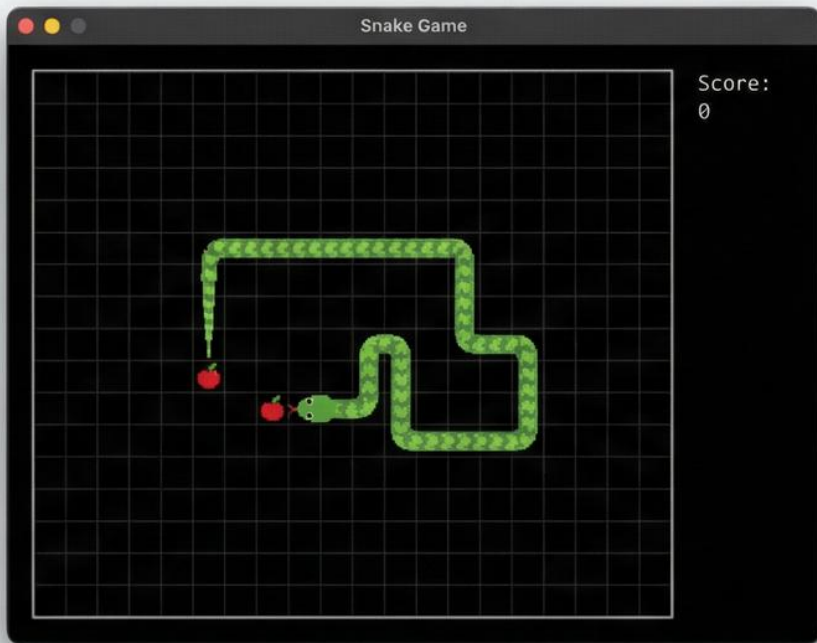
Por que Python? - Sintaxe Simples!

- A linguagem foi projetada para ter uma leitura fácil e intuitiva.
- Não há a obrigatoriedade ou necessidade de declarar um método principal para iniciar a execução, como o clássico 'main()' presente em C ou Java.
- O Python utiliza uma indentação forçada. Essa característica não apenas define os blocos de comando, mas também provoca e educa o programador a manter uma excelente organização visual do código.

A window titled "Python Code Example" with a blue header and yellow title bar. It contains a Python code snippet with line numbers 1 through 9 on the left. The code demonstrates simple syntax with input, print, and an if-else statement.

```
1 # Exemplo de Sintaxe Simples
2 nome = input("Qual é o seu nome? ")
3 idade = int(input("Qual é a sua idade? "))
4
5 print(f"Olá, {nome}!")
6
7 if idade >= 18:
7     print("Você é maior de idade.")
8 else:
9     print("Você é menor de idade.")
```

Por que Python? - Resultados em Pouco Tempo



-  - O Python é uma linguagem extremamente completa, contando com uma ampla gama de bibliotecas nativas e de terceiros para solucionar problemas rapidamente.
-  - Suas bibliotecas facilitam tarefas complexas, como acessar bancos de dados, processar arquivos estruturados (XML ou JSON), construir interfaces gráficas (GUI) e até mesmo programar games.
-  - Toda essa infraestrutura pronta aumenta consideravelmente a produtividade do programador, reduzindo o tempo de desenvolvimento.
-  - Além disso, a linguagem possui uma imensa e acolhedora comunidade, o que garante abundância em material de estudo e fóruns de suporte disponíveis.

Por que Python? - Muitos Frameworks

Um framework é um conjunto de ferramentas que facilita a criação de aplicações, oferecendo uma base pronta e segura para o desenvolvimento. Python brilha no desenvolvimento web e de APIs, oferecendo opções robustas para todos os tamanhos de projetos.



Django Web



Flask



web2py



Bottle

Alguns dos frameworks mais conhecidos e utilizados no mercado incluem: Django Web, Flask, FastAPI, CherryPy, Bottle, Pyramid, Falcon e Dash. Existem ainda centenas de outras ferramentas específicas para análise de dados, machine learning e automação.

Como utilizar o Python? - O Interpretador

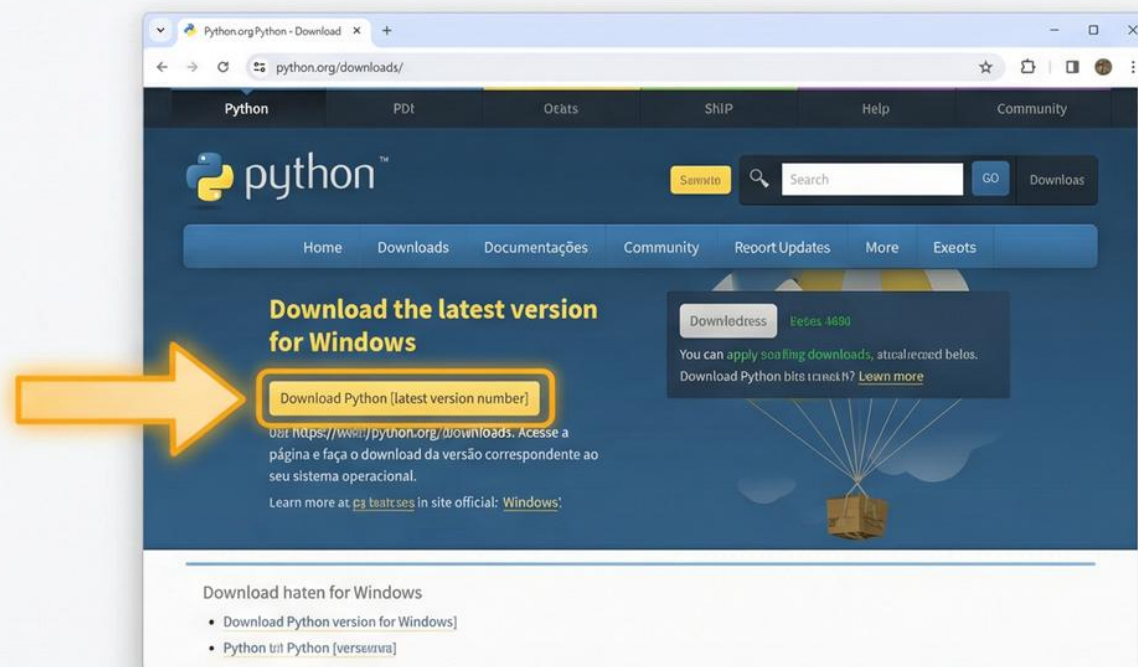


- Python é uma linguagem interpretada. Isso significa que você precisa de um Interpretador Python instalado na sua máquina.
- O interpretador é o programa responsável por ler, entender e executar os comandos que você digita, permitindo testar trechos de código e verificar os resultados instantaneamente.
- Sua facilidade de uso tem atraído um público gigantesco: a cada dia que passa, mais e mais pessoas, de diversas áreas, estão aprendendo Python e tornando a linguagem um padrão de mercado.

Instalação do Python - Download

O primeiro passo para programar em Python é baixar o instalador oficial diretamente do site da fundação. Todas as versões, atualizações e documentações do Python estão disponíveis gratuitamente e de forma segura no site oficial:

<https://www.python.org/downloads/>. Acesse a página e faça o download da versão correspondente ao seu sistema operacional. Geralmente, o site reconhece seu sistema e disponibiliza a versão recomendada (como a de 64 bits para Windows) em um grande botão amarelo logo na primeira página.

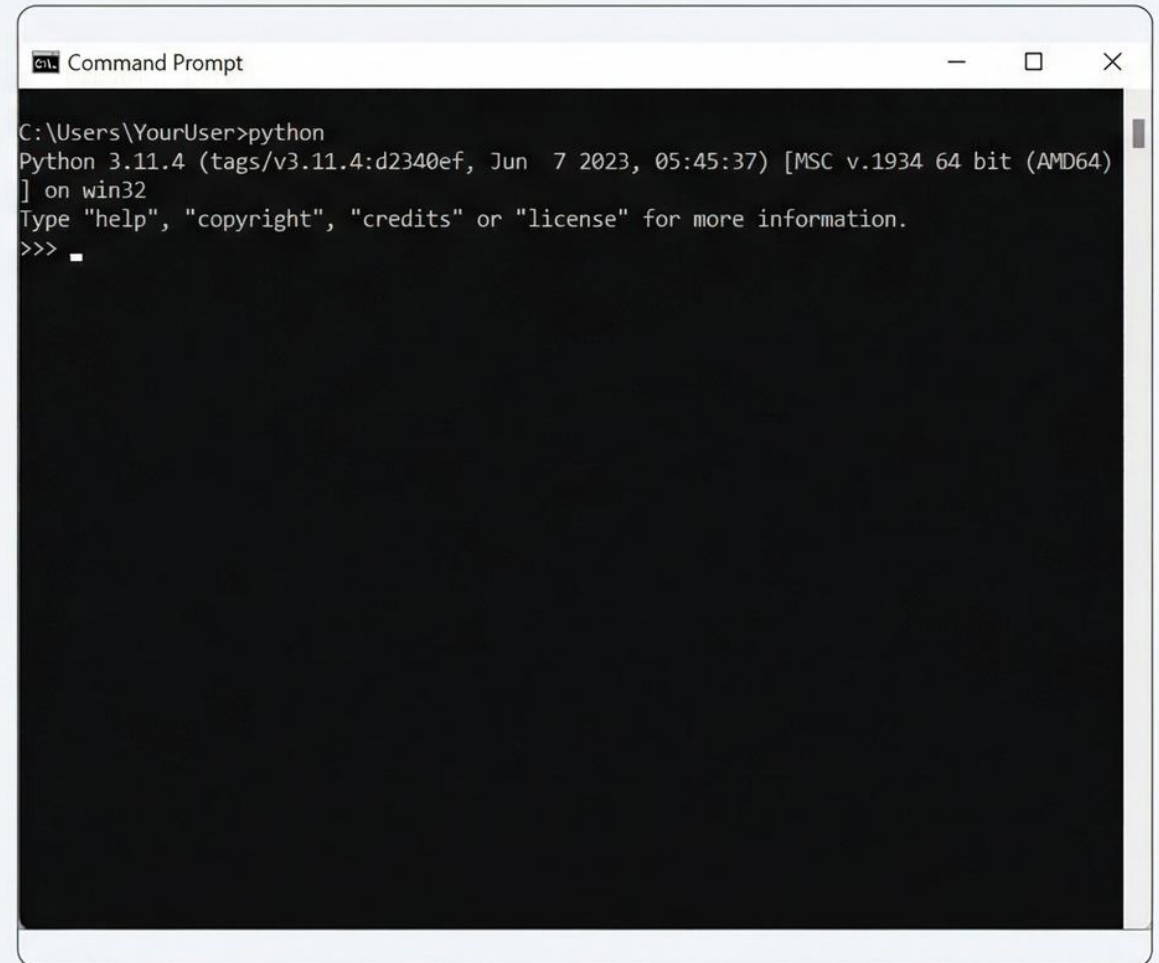


Instalação do Python - Prompt de Comando

Após seguir o processo simples de instalação (lembrando sempre de marcar a opção 'Add Python to PATH'), o ambiente estará pronto!

Pronto! Seu Python está devidamente instalado e pronto para ser utilizado no seu sistema.

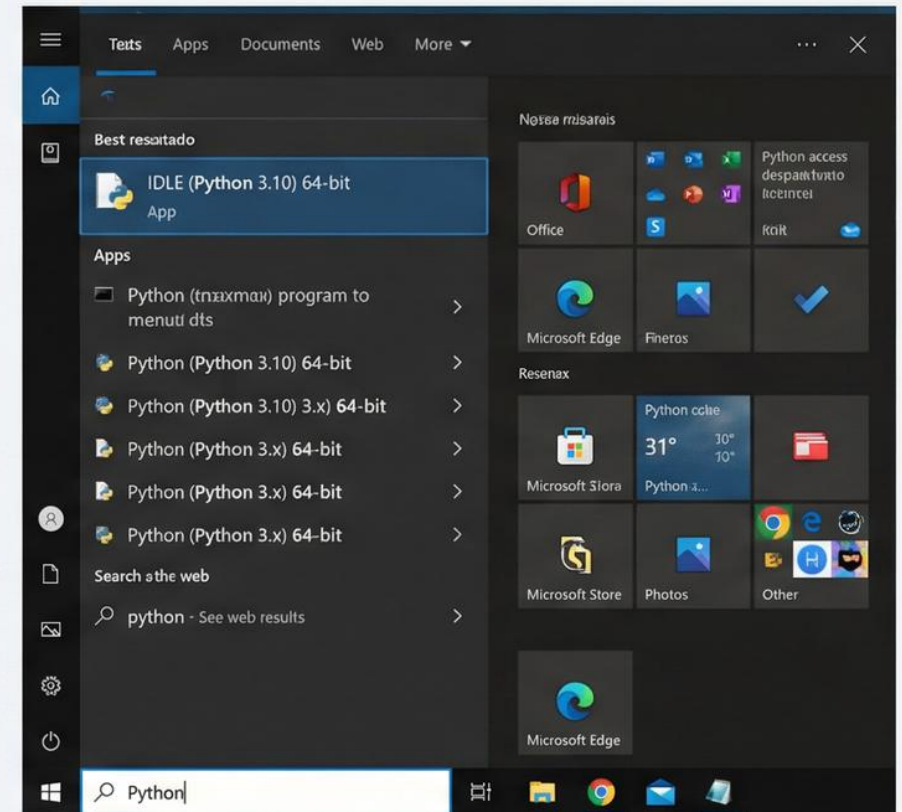
A primeira forma de testá-lo é utilizando o ambiente de linha de comando padrão do sistema: abra o Prompt de Comando do Windows (conhecido por CMD). Com a tela preta do prompt aberta, basta digitar a palavra 'python' (tudo em minúsculo) e teclar Enter. O interpretador será iniciado e você verá a versão do Python instalada seguida pelo sinal '>>>'.
>>>



```
Command Prompt
C:\Users\YourUser>python
Python 3.11.4 (tags/v3.11.4:d2340ef, Jun 7 2023, 05:45:37) [MSC v.1934 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> █
```

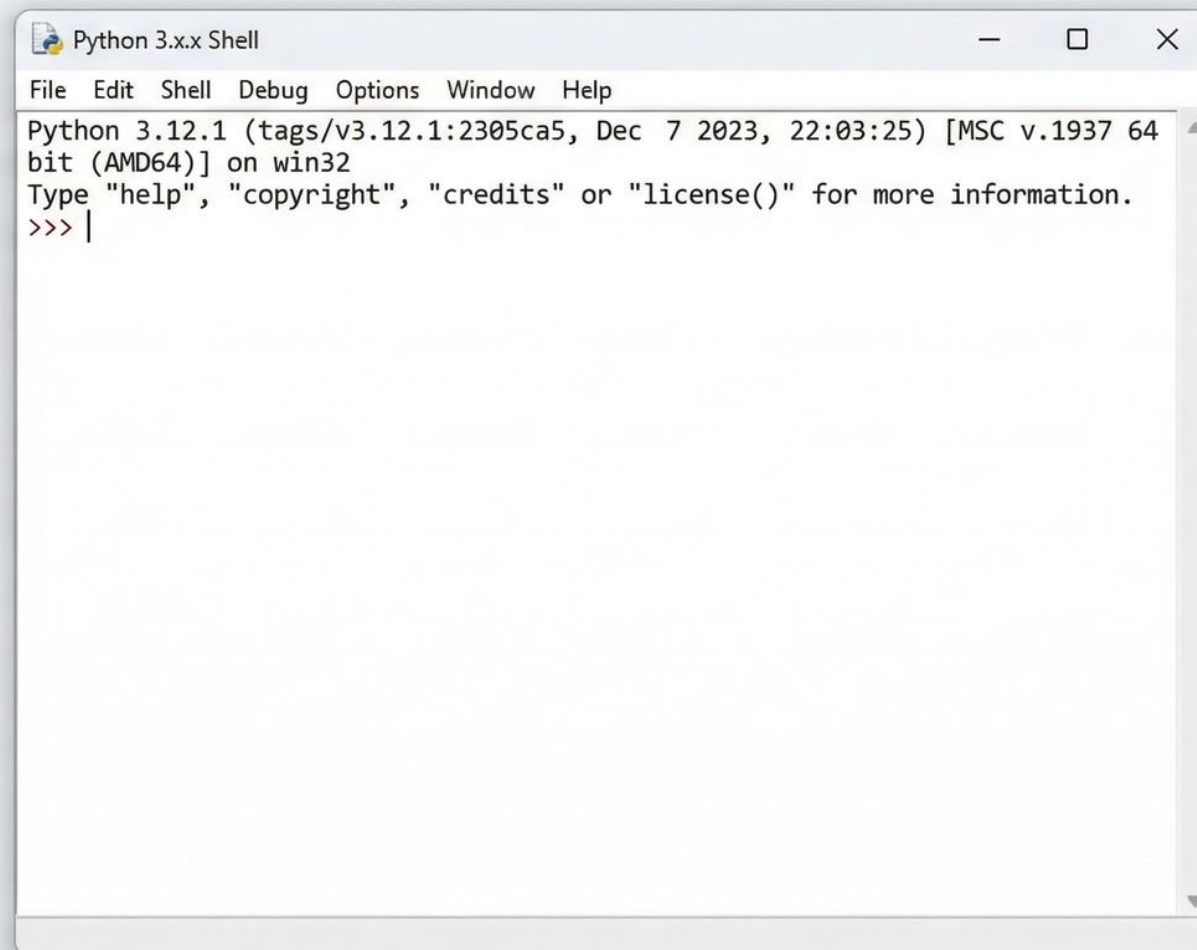

Instalação do Python - O IDLE (Parte 1)

- Embora seja possível programar pelo CMD, o Python já vem com um ambiente de desenvolvimento integrado básico, chamado IDLE.
- O IDLE é perfeito para iniciantes, pois possui interface gráfica, destaque de sintaxe em cores e ferramentas embutidas de fácil manuseio.
- Para abri-lo pela primeira vez, vá até a barra de pesquisa ou menu iniciar do seu sistema e digite 'Python'.
- Nos resultados da busca, procure pelo aplicativo chamado 'IDLE (Python 3.x)' — onde o 'x' é a versão instalada no seu computador — e clique para executar.



Instalação do Python - O IDLE (Parte 2)

- Ao clicar no ícone, o ambiente IDLE iniciará a sua janela principal, que é o Shell interativo.
- Nesta janela, você verá uma tela branca com informações sobre a versão do Python na primeira linha e o cursor piscando aguardando instruções.
- O Shell do IDLE funciona de forma muito semelhante ao prompt de comando, executando linha por linha, mas conta com opções extras no menu superior (**File**, **Edit**, **Shell**, **Debug**, etc).
- É neste ambiente que executaremos nossos primeiros testes e comandos.



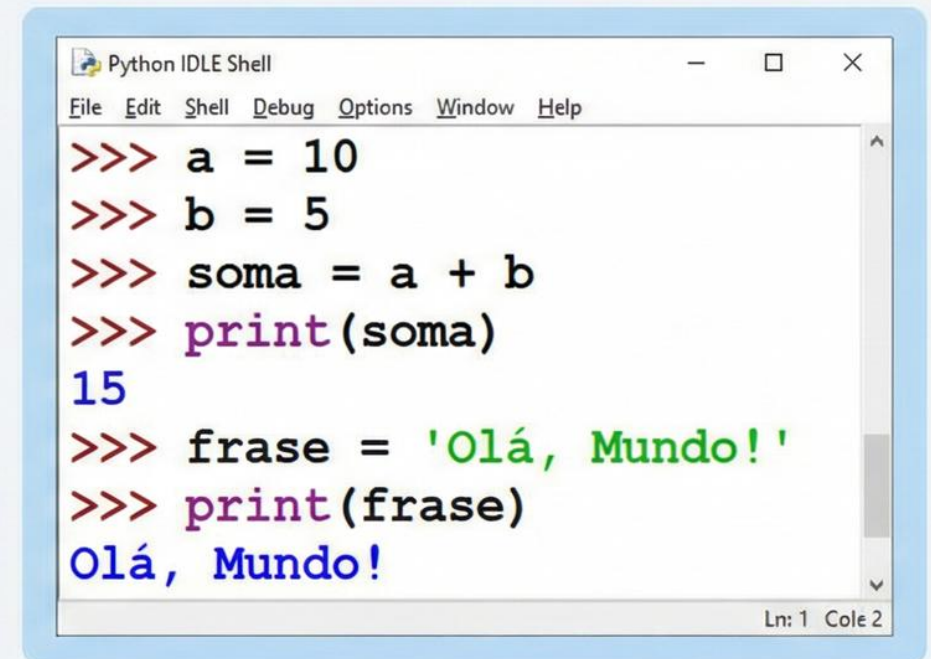
Conceitos Básicos: Variáveis

Variáveis são espaços reservados na memória do computador para armazenar dados que serão utilizados pelo seu programa.

Em Python, a criação de variáveis é dinâmica. Não precisamos declarar o tipo da variável antes de utilizá-la.

As variáveis recebem seu valor por meio de uma operação de atribuição, que é feita utilizando o sinal de igualdade '='.

A estrutura padrão é: nome_da_variavel = valor_a_ser_armazenado.

A screenshot of the Python IDLE Shell window. The window has a title bar that says "Python IDLE Shell" and a menu bar with "File", "Edit", "Shell", "Debug", "Options", "Window", and "Help". The main text area contains the following code:

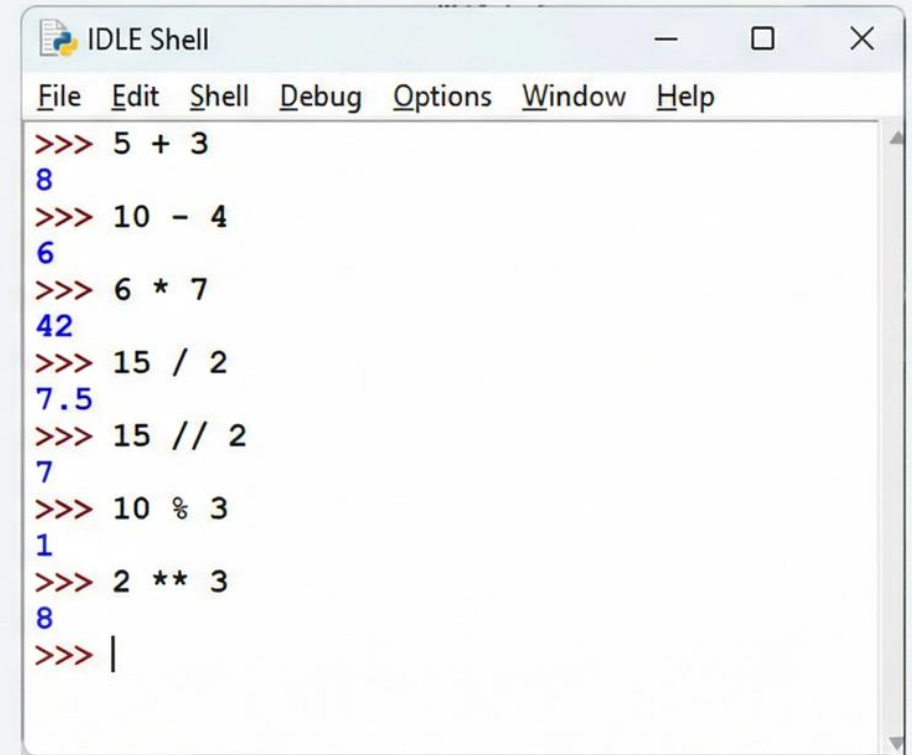
```
>>> a = 10
>>> b = 5
>>> soma = a + b
>>> print(soma)
15
>>> frase = 'Olá, Mundo!'
>>> print(frase)
Olá, Mundo!
```

 The output "15" and "Olá, Mundo!" are displayed in blue. The status bar at the bottom right shows "Ln: 1 Cole 2".

Operadores Aritméticos

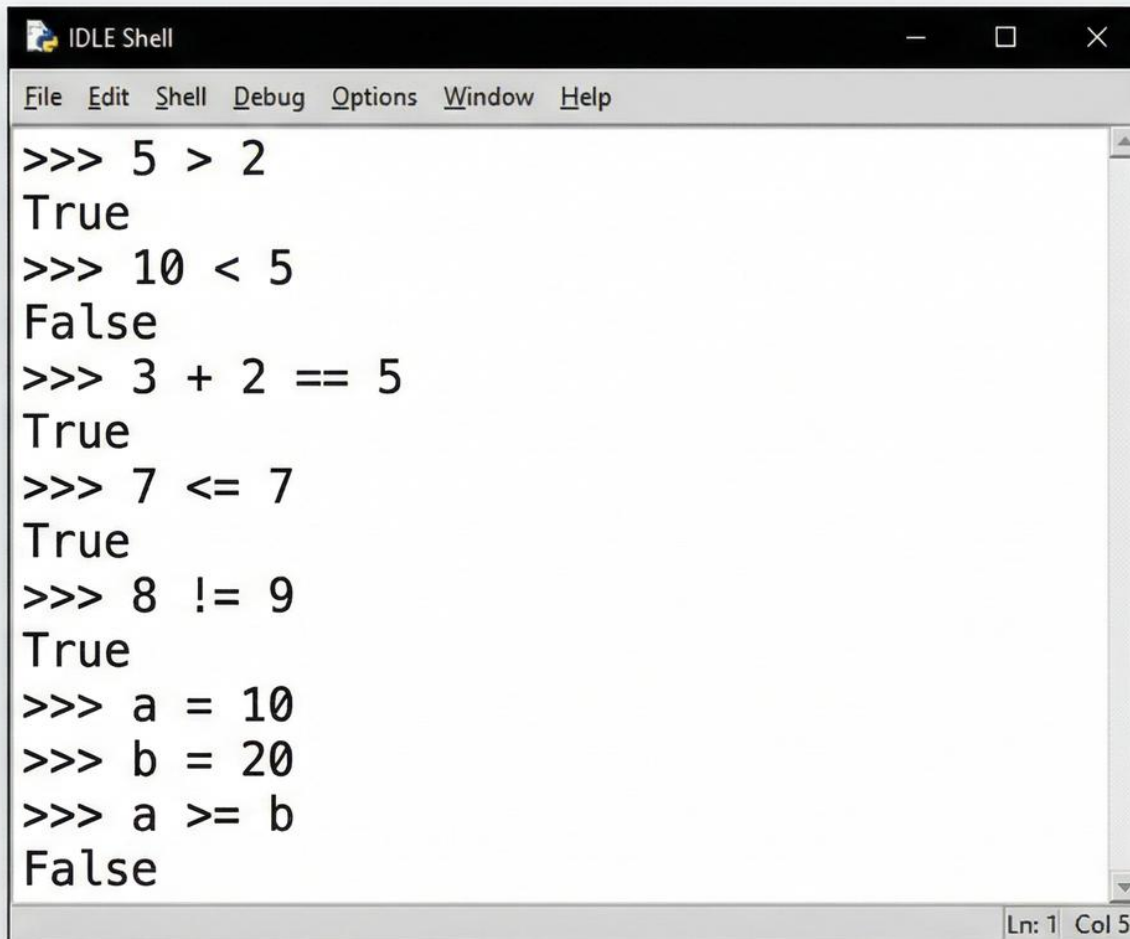
Os operadores aritméticos são utilizados em conjunto com variáveis ou valores diretos para realizar operações matemáticas comuns.

- (+) Adição: Soma dois valores.
- (-) Subtração: Subtrai um valor do outro.
- (*) Multiplicação: Multiplica dois valores.
- (/) Divisão: Realiza a divisão normal, retornando um número decimal (float).
- (//) Divisão Inteira: Realiza a divisão e descarta a parte fracionária.
- (%) Módulo: Retorna o resto de uma divisão.
- (**) Exponenciação: Eleva um número à potência de outro.



```
IDLE Shell
File Edit Shell Debug Options Window Help
>>> 5 + 3
8
>>> 10 - 4
6
>>> 6 * 7
42
>>> 15 / 2
7.5
>>> 15 // 2
7
>>> 10 % 3
1
>>> 2 ** 3
8
>>> |
```


Operadores Relacionais



```
IDLE Shell
File Edit Shell Debug Options Window Help
>>> 5 > 2
True
>>> 10 < 5
False
>>> 3 + 2 == 5
True
>>> 7 <= 7
True
>>> 8 != 9
True
>>> a = 10
>>> b = 20
>>> a >= b
False
Ln: 1 Col 5
```

- Os operadores relacionais servem para realizar comparações entre dois valores ou variáveis.
- O resultado de uma comparação relacional sempre será um valor Booleano: Verdadeiro (True) ou Falso (False).

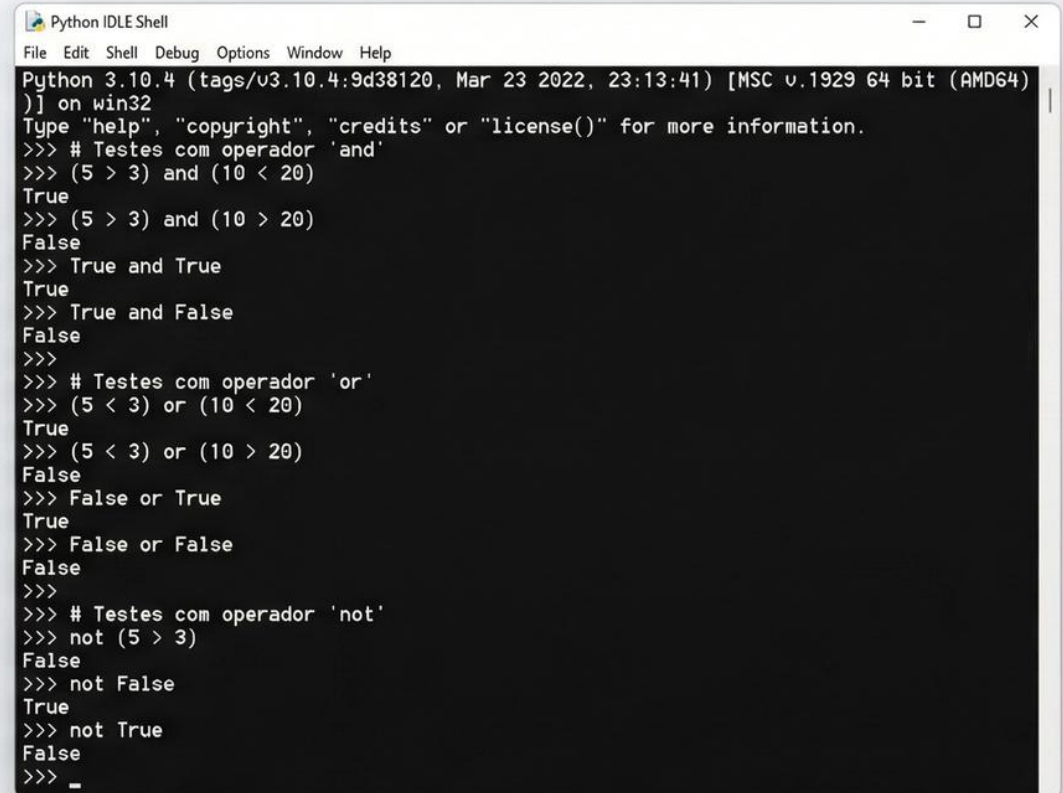
Lista de operadores relacionais em Python:

• (<)	Menor que
• (>)	Maior que
• (<=)	Menor ou igual a
• (>=)	Maior ou igual a
• (==)	Igual a (atenção: um único igual '=' é atribuição, duplo '==' é comparação)
• (!=)	Diferente de

Operadores Lógicos

Os operadores lógicos permitem combinar múltiplas comparações relacionais ou alterar o valor lógico de uma expressão. Em Python, utilizamos palavras-chave do idioma inglês (em minúsculas) para representar essas lógicas:

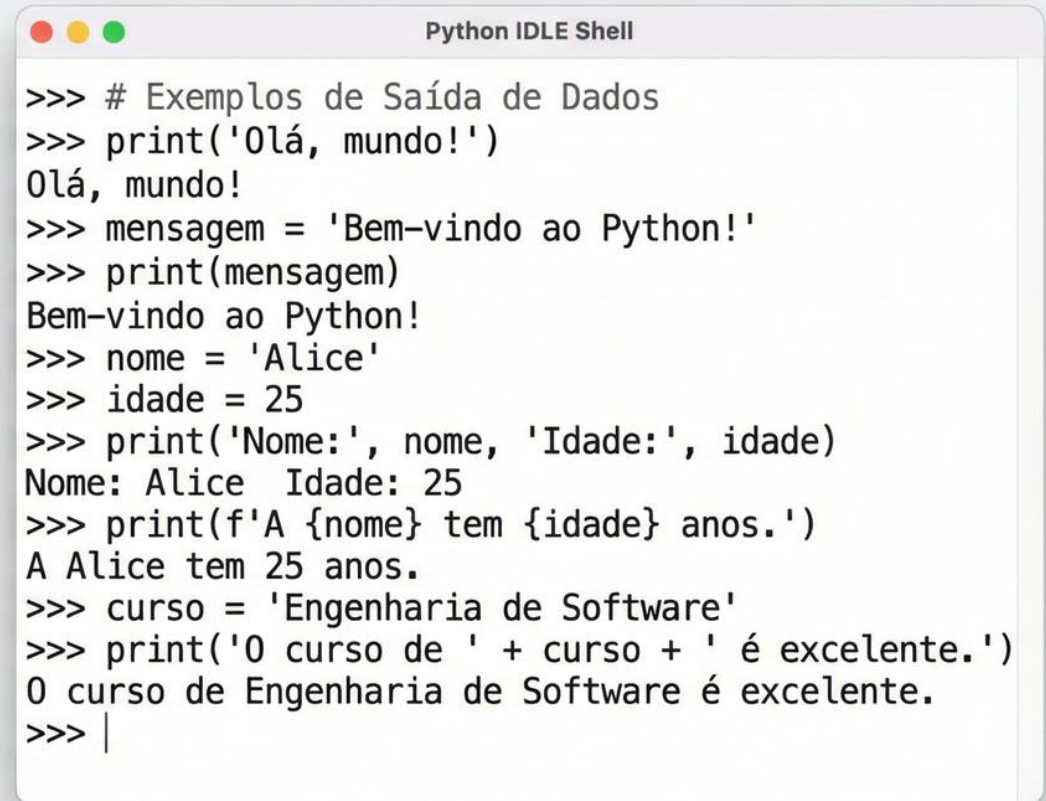
- **and** : Equivalente ao 'E' lógico. Retorna True apenas se TODAS as condições forem verdadeiras.
- **or** : Equivalente ao 'OU' lógico. Retorna True se PELO MENOS UMA das condições for verdadeira.
- **not** : Equivalente à negação ('NÃO'). Inverte o resultado booleano de uma expressão (o que é True vira False, e vice-versa).

A screenshot of a Python IDLE Shell window. The window has a title bar 'Python IDLE Shell' and a menu bar with 'File', 'Edit', 'Shell', 'Debug', 'Options', 'Window', and 'Help'. The main text area shows the following code and output:

```
Python 3.10.4 (tags/v3.10.4:9d38120, Mar 23 2022, 23:13:41) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> # Testes com operador 'and'
>>> (5 > 3) and (10 < 20)
True
>>> (5 > 3) and (10 > 20)
False
>>> True and True
True
>>> True and False
False
>>>
>>> # Testes com operador 'or'
>>> (5 < 3) or (10 < 20)
True
>>> (5 < 3) or (10 > 20)
False
>>> False or True
True
>>> False or False
False
>>>
>>> # Testes com operador 'not'
>>> not (5 > 3)
False
>>> not False
True
>>> not True
False
>>> _
```


Saída de Dados

- Para que o programa interaja com o usuário exibindo resultados e mensagens na tela, utilizamos os comandos de saída.
- No Python, a saída padrão de dados acontece através da função integrada `print()`.
- Podemos imprimir mensagens de texto (strings) envolvendo-as em aspas, como: `print('Olá, mundo!')`
- Também podemos exibir os valores de uma variável diretamente, ou misturar textos com variáveis utilizando vírgulas ou técnicas de formatação para concatenar as informações de maneira legível.

A screenshot of a Python IDLE Shell window. The window has a title bar with three colored buttons (red, yellow, green) on the left and the text "Python IDLE Shell" on the right. The main area contains a series of Python commands and their outputs. The commands are: a comment "# Exemplos de Saída de Dados", `print('Olá, mundo!')`, `mensagem = 'Bem-vindo ao Python!'`, `print(mensagem)`, `nome = 'Alice'`, `idade = 25`, `print('Nome:', nome, 'Idade:', idade)`, `print(f'A {nome} tem {idade} anos.')`, `curso = 'Engenharia de Software'`, and `print('O curso de ' + curso + ' é excelente.')`. The outputs are: "Olá, mundo!", "Bem-vindo ao Python!", "Nome: Alice Idade: 25", "A Alice tem 25 anos.", and "O curso de Engenharia de Software é excelente.". The prompt `>>>` is visible at the end of each line of code.

```
>>> # Exemplos de Saída de Dados
>>> print('Olá, mundo!')
Olá, mundo!
>>> mensagem = 'Bem-vindo ao Python!'
>>> print(mensagem)
Bem-vindo ao Python!
>>> nome = 'Alice'
>>> idade = 25
>>> print('Nome:', nome, 'Idade:', idade)
Nome: Alice Idade: 25
>>> print(f'A {nome} tem {idade} anos.')
A Alice tem 25 anos.
>>> curso = 'Engenharia de Software'
>>> print('O curso de ' + curso + ' é excelente.')
O curso de Engenharia de Software é excelente.
>>> |
```

Entrada de Dados

- A maioria dos programas precisa receber informações que o usuário digita no teclado durante a execução.
- Para capturar esses dados inseridos pelo usuário e atribuí-los a uma variável, utilizamos a função embutida `input()`.
- A função `input()` pode receber uma mensagem como parâmetro para orientar o usuário sobre o que ele deve digitar. Por exemplo: `nome = input('Digite seu nome: ')`
- **Atenção:** A função `input()` SEMPRE lê o dado como um texto (string), mesmo que você digite números. 🤔

```
>>> nome = input('Digite seu nome: ')
Digite seu nome: Maria
>>> print(type(nome))
<class 'str'> 👍
>>> idade = input('Digite sua idade: ')
Digite sua idade: 25
>>> print(type(idade))
<class 'str'> 🤔
>>> idade_num = int(input('Digite sua idade novamente: '))
Digite sua idade novamente: 25
>>> print(type(idade_num))
<class 'int'> 👍
```


Conversão de Tipos (Casting)

- Como a função `input()` retorna textos por padrão, precisamos converter esse texto em números caso a intenção seja realizar cálculos matemáticos.
- Uma vez tendo um determinado valor capturado em uma variável, é possível convertê-la (fazer o casting) utilizando funções construtoras do tipo desejado.
 - Para converter um texto em número inteiro, utilizamos a função `int()`. Ex: `int('10')`
 - Para converter um texto em número decimal, utilizamos a função `float()`. Ex: `float('3.14')`

Essa técnica é fundamental em lógicas que exigem precisão matemática, como médias de notas, cálculos de idade e medidas financeiras.

```
Python 3.x.x Shell

>>> # Cálculo do Índice de Massa Corporal (IMC)
>>> peso_str = input('Digite seu peso (kg): ')
Digite seu peso (kg): 70.5
>>> altura_str = input('Digite sua altura (m): ')
Digite sua altura (m): 1.75

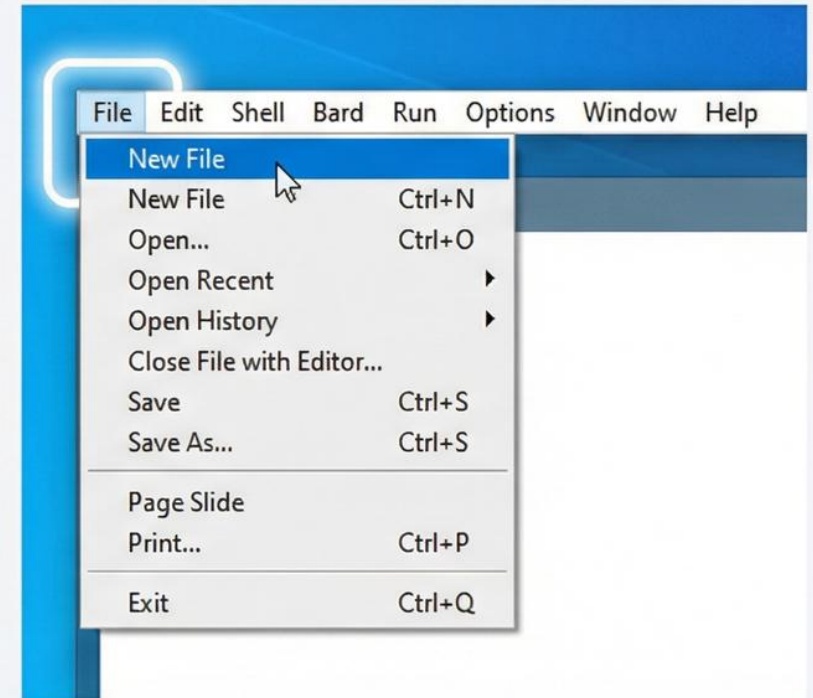
>>> # Convertendo as entradas para números decimais (float)
>>> peso = float(peso_str)
>>> altura = float(altura_str)

>>> # Convertendo uma entrada para número inteiro (int) -
exemplo adicional
>>> idade_str = input('Digite sua idade: ')
Digite sua idade: 30
>>> idade = int(idade_str)

>>> # Calculando o IMC
>>> imc = peso / (altura * altura)
>>> print('Seu IMC é:', imc)
Seu IMC é: 23.020408163265305
>>> print('Sua idade é:', idade)
Sua idade é: 30
```

Nosso Primeiro Programa - Criando o Arquivo

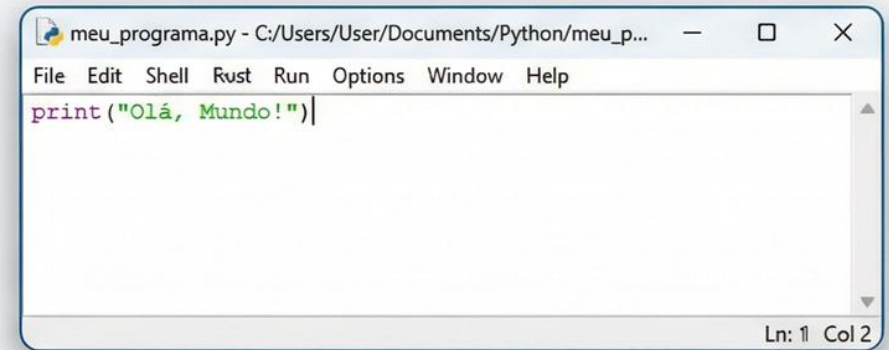
- Trabalhar no Shell interativo é ótimo para testes rápidos, mas os comandos são perdidos ao fecharmos a janela.
- Para elaborar um programa real e salvar o código fonte de maneira definitiva, precisamos escrever em um arquivo de script.
- Através do IDLE, criar um script é muito simples: basta ir na barra superior de menus, clicar em "File" (Arquivo) e, em seguida, selecionar "New File" (Novo Arquivo) ou simplesmente usar o atalho Ctrl+N.
- Isso abrirá uma janela em branco que funcionará como um editor de textos focado em código Python.



Nosso Primeiro Programa - Executando o Código

Roboto Bold

- Com o editor de textos do IDLE aberto e vazio, é o momento de programar de verdade.
- Digite as instruções do seu algoritmo, linha por linha, respeitando a lógica desejada.
- Após concluir a digitação, é crucial salvar o seu trabalho no computador (File > Save) com a extensão ".py".
- Uma vez salvo, execute o seu programa! Basta ir ao menu superior em "Run" e clicar na opção "Run Module", ou utilizar a tecla de atalho rápida "F5".
- Ao fazer isso, o IDLE transferirá a execução para a tela do **Shell**, onde as interações com o usuário acontecerão.

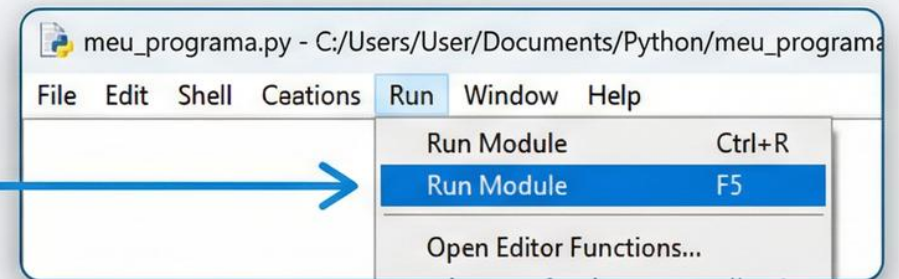


meu_programa.py - C:/Users/User/Documents/Python/meu_p... — □ ×

File Edit Shell Rust Run Options Window Help

```
print("Olá, Mundo!")
```

Ln: 1 Col 2



Olá, Mundo!



Desafio Prático 1

Agora é a hora de colocar o conhecimento em prática, estruturando um algoritmo simples para fixar os conceitos abordados na aula de hoje.

- **O problema a ser solucionado é:** Elabore um programa em Python que peça para o usuário inserir dois números inteiros. Em seguida, o programa deve realizar o processamento da soma desses dois valores e, por fim, imprimir o resultado final formatado na tela.

Desafio Prático 2

Subindo um pouco a complexidade, nosso segundo desafio envolverá o uso de tipos variados de dados (texto e decimais). O problema a ser solucionado é: Elabore um programa que solicite, primeiramente, o nome de um aluno. Na sequência, peça para o usuário inserir duas notas diferentes deste aluno. O programa deve então calcular a média aritmética do aluno e, por fim, exibir o nome do aluno junto com a média final obtida nas avaliações.